EE120B Custom Lab Project Report

Dinosaur Game

Brett Castro
6/11/2025

Introduction

The Google dinosaur game is an original game developed by Google built into the Chrome browser. The game is played using a joystick input to jump and to duck. The goal of the game is to jump and duck to avoid obstacles that appear while the dinosaur runs forward. As the dinosaur progresses, the game speeds up, with a score accruing for time spent alive. The player loses the game when they make contact with an obstacle. The game runs until the player loses, continually growing in speed, and thus difficulty. The high score of the game is saved and can be reset with a button press. The game can be turned off with another button, putting the arduino in sleep mode and turning off the displays.

Build-Upons

The first build upon is the ST7735 display. This is the main display that shows gameplay, including the dinosaur and its jump/duck animations, the cactus and pterodactyl moving forward, and the collision animation.

The second build upon is the 16x2 display included in the Elegoo kit. This display handles the outputting of high and current scores and the “Press Start” and “Game Over” text.

The third build upon is the EEPROM memory in the arduino. This was implemented to save the high score locally between playthroughs, and can be reset to 0.

The fourth build upon is the sleep and interrupt abilities in the arduino. This was implemented so that when the power button is pressed, the CPU sleeps, waiting for the button to be pressed again to wake the system.

The fifth build upon is the use of the L293D motor driver. This chip was utilized to control the backlighting of the displays. By connecting the LED pins on the display to the chip, pressing the power button will turn the displays off and on, synchronized with the CPU sleeping.

All build-upons are fully functional.

User Guide

When connected to power, the system automatically turns on and is ready for gameplay. On the main display, a dinosaur and cactus appear. On the secondary LCD, the user will be prompted to press the start button to start a game. Underneath that text is the current high score.

To start a game, the user presses down on the joystick. When a game is started, moving the joystick up causes the dinosaur to jump, and by moving the joystick down, the dinosaur ducks. The user will encounter 2 types of obstacles; cactus and pterodactyl. The obstacles will move towards the dinosaur; if a cactus appears, the user must jump over it, and if a pterodactyl appears, the user is able to jump over or duck under it.

The right button is the power button. Pressing this button when the system is on will end a game if one is active, and will set the processor CPU to sleep mode, while also turning off the displays. Pressing the button when the system is off will turn both displays back on, resetting them to the initial game state, and will wake the CPU.

The left button is the reset button. Pressing this button while a game is active will do nothing. When a game is not active, pressing this button will set the high score to 0.

Hardware Components

Computing

- Elegoo Uno R3

Inputs

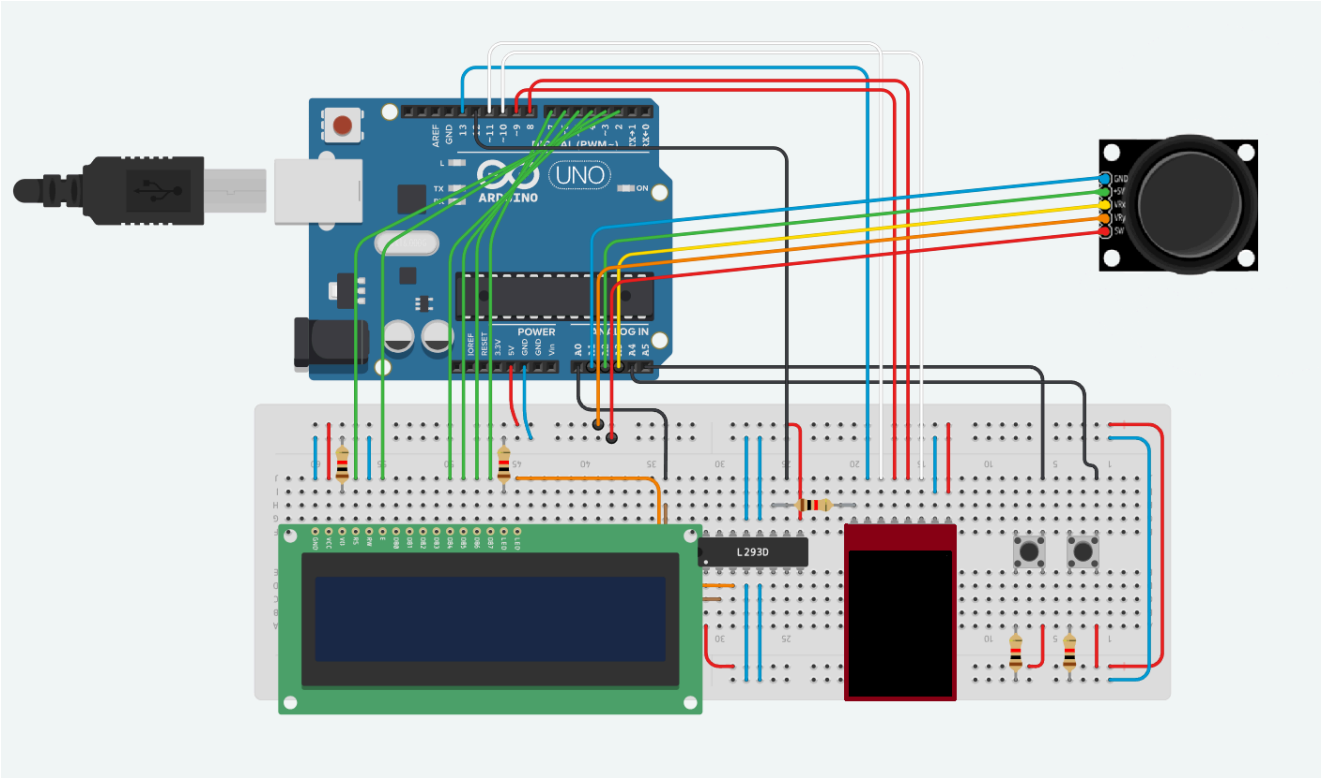
- Joystick
- Buttons

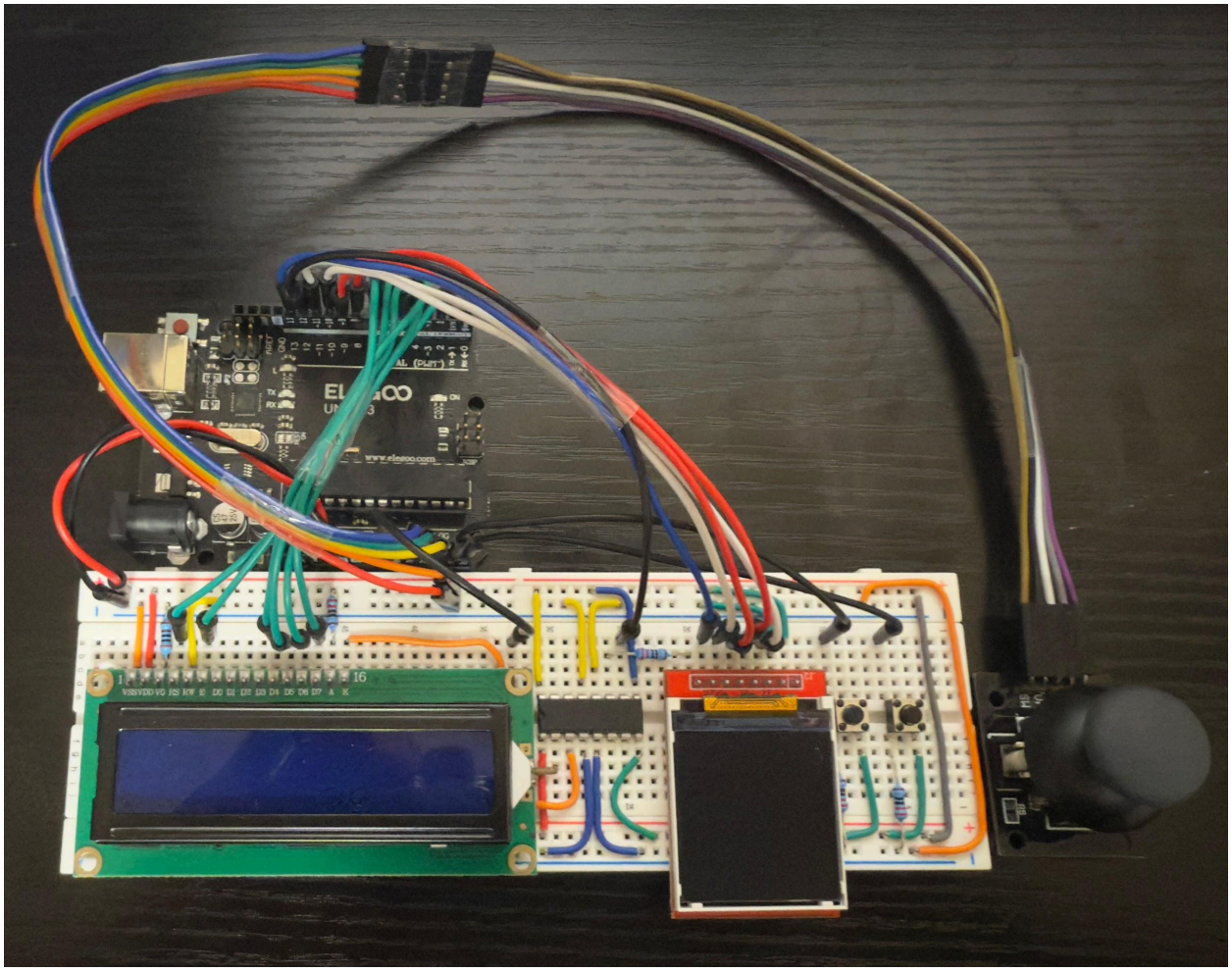
Outputs

- LCD1602 screen (Kit)
- HiLetgo ST7735 LCD screen (Bought)
- L293D Motor Driver (LEDs)

Software Libraries

- stdio.h
    - This library was used for the snprintf function. Using this function, text can be merged with an int into a string. This allowed 2 strings to be used for output to the LCD for the scores.
- string.h
    - This library was used for the strcpy function, which was used for the text that did not include a numerical value.
- stdlib.h
    - This library was used for the rand function, which allowed for the implementation of random obstacle generation. Using this function, a 70/30% split chance of getting a cactus vs a pterodactyl exists.
- avr/eeprom.h
    - This library was used to implement the saved high score. The included eeprom_read_word and eeprom_update_word functions are utilized to read and update the saved score.
- avr/sleep.h
    - This library was used to implement the power off functionality. Using the set_sleep_mode, sleep_enable, sleep_cpu, and sleep_disable functions allowed the CPU to be powered down to a low energy state.
- avr/interrupt.h
    - This library was used to finish implementation of the power off functionality. Detecting an interrupt from the pin connected to the power button when the system was powered off allowed for the system to be woken up.

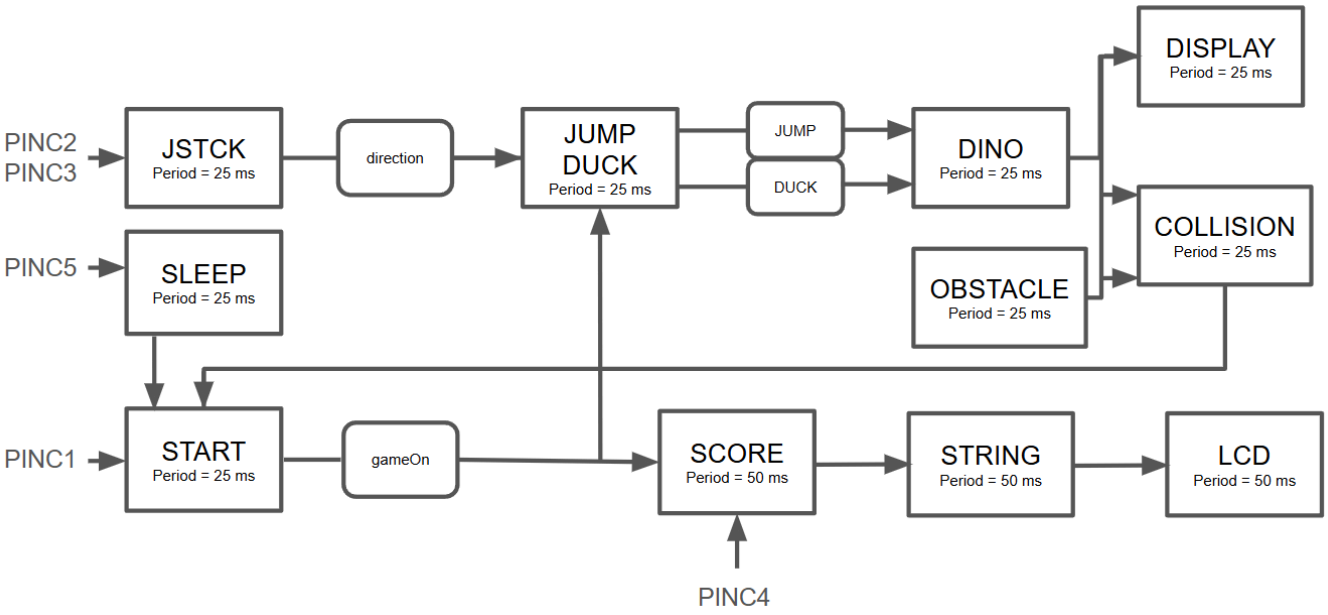

UNO
ARDUINO
POWER
ANALOG IN
L293D
GND
+5V
Vrx
Vry
Sw

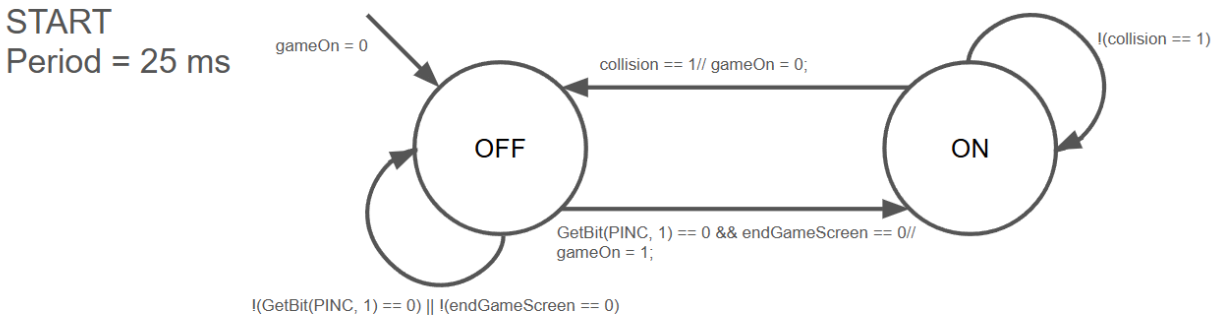


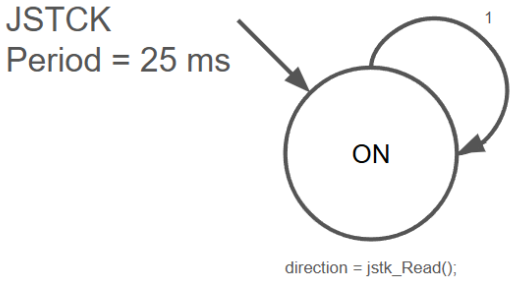
GLOBAL VARIABLES

```
unsigned char direction; // 0 = - | 2 = U | 1 = D | 3 = L | 4 = R (used in JUMP_DUCK task)
unsigned char jump;      // Flag when a jump is detected (used in DINO task)
unsigned char duck;      // Flag when a duck is detected (used in DINO task)

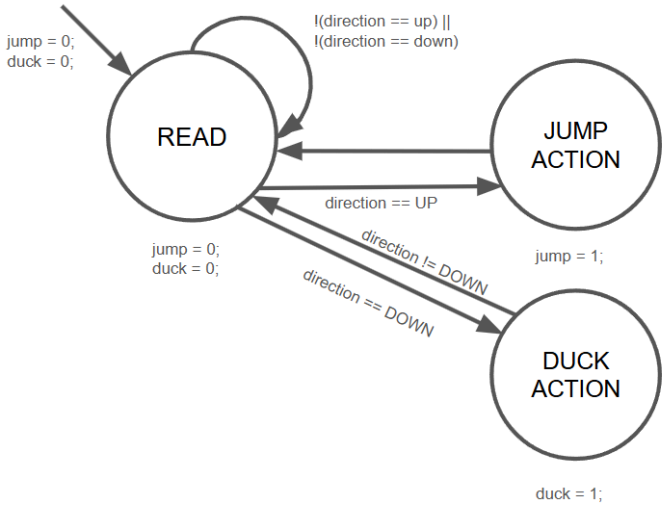
//Global Flags used throughout tasks to transition between game states
unsigned char collision; // Flag when a collision is detected
unsigned char gameOn;    // Flag when a start button press is detected
unsigned char endGameScreen; // Flag when the end game screen is finished being displayed
unsigned char sleeping; // Flag when the sleep button is pressed
```


PINC2
PINC3
JSTCK
Period = 25 ms
direction
JUMP
DUCK
Period = 25 ms
JUMP
DUCK
DINO
Period = 25 ms
DISPLAY
Period = 25 ms
COLLISION
Period = 25 ms
OBSTACLE
Period = 25 ms
PINC5
SLEEP
Period = 25 ms
PINC1
START
Period = 25 ms
gameOn
SCORE
Period = 50 ms
STRING
Period = 50 ms
LCD
Period = 50 ms
PINC4

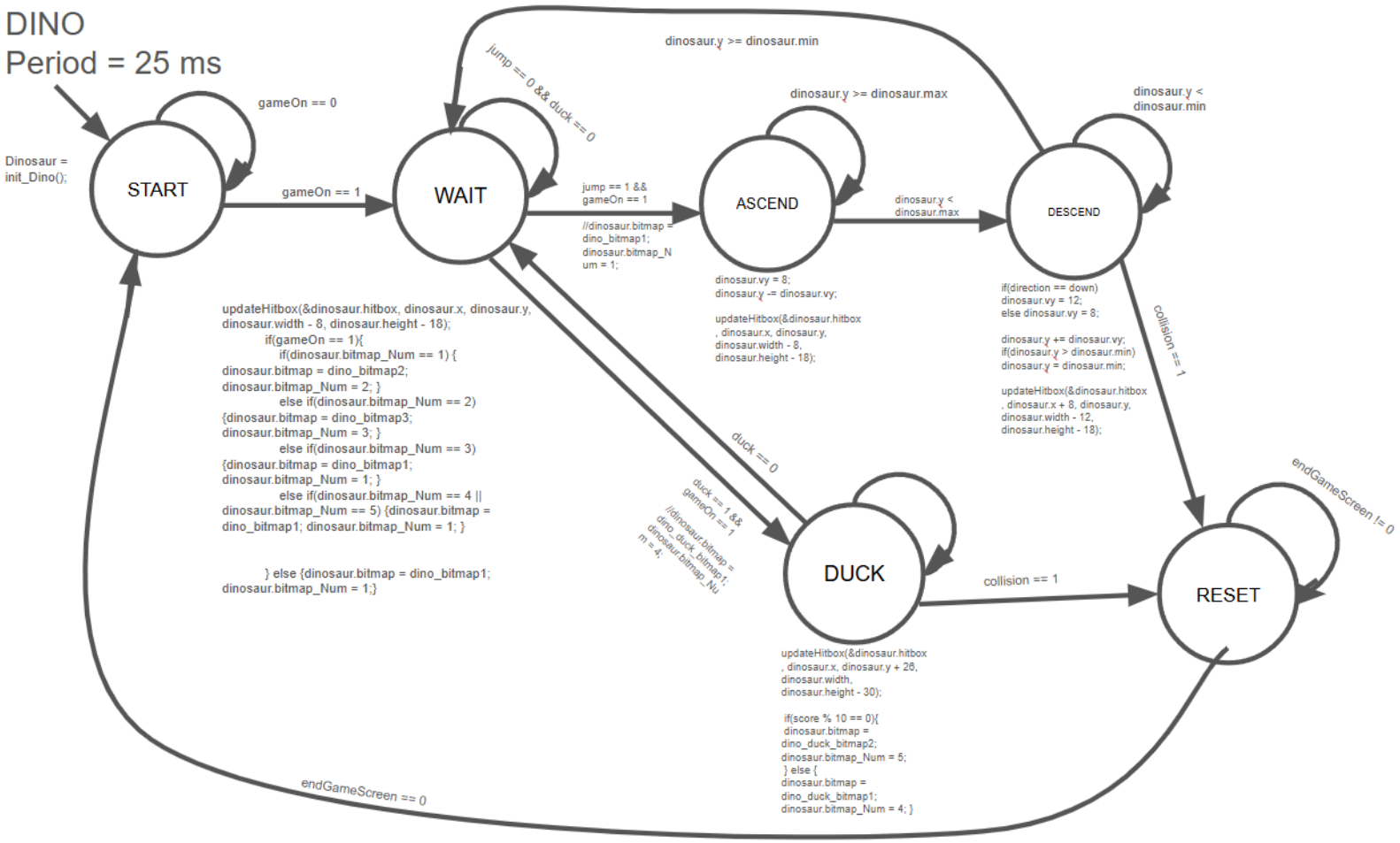

START
Period = 25 ms
gameOn = 0
OFF
collision == 1// gameOn = 0;
ON
!(collision == 1)
GetBit(PINC, 1) == 0 && endGameScreen == 0//
gameOn = 1;
!(GetBit(PINC, 1) == 0) || !(endGameScreen == 0)


JSTCK
Period = 25 ms
1
ON
direction = jstk_Read();

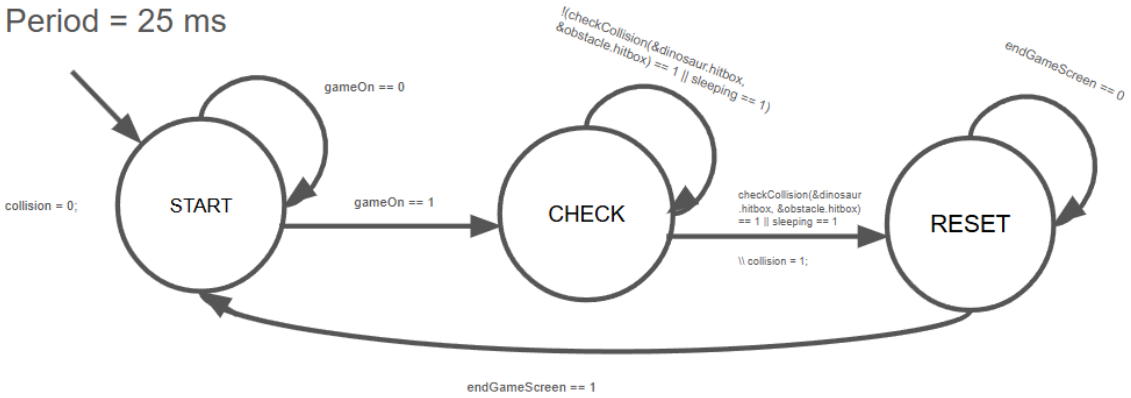
JUMP_DUCK
Period = 25 ms



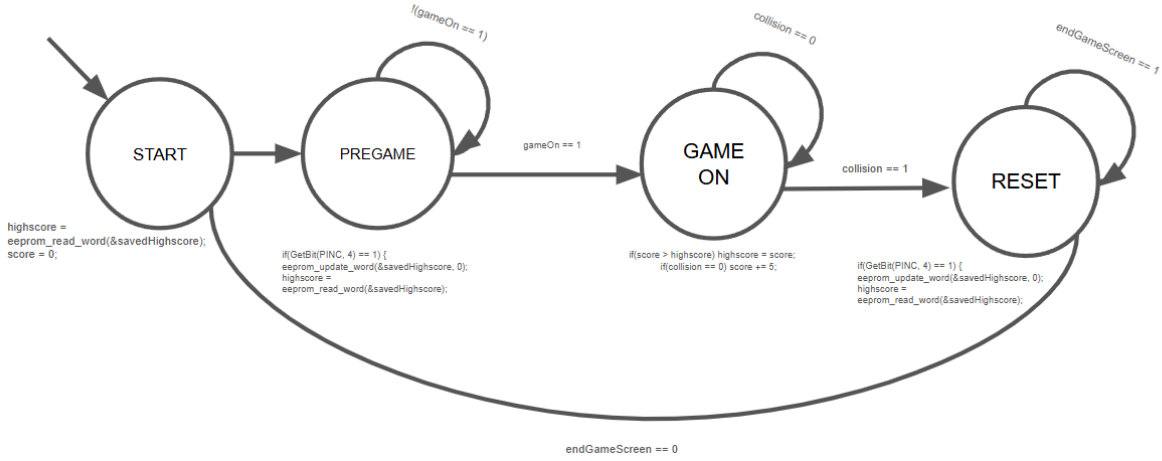
DINO
Period = 25 ms



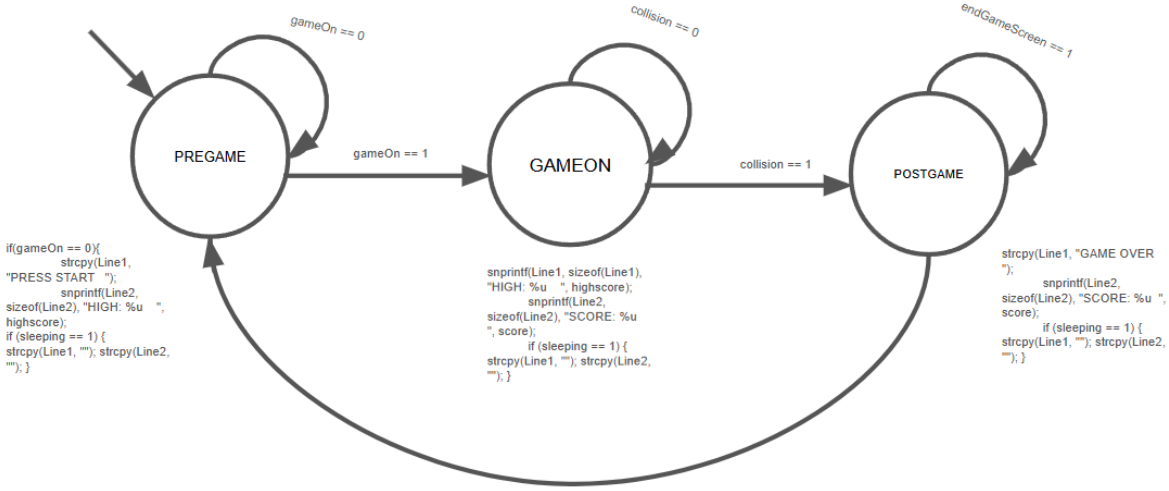
COLLISION
Period = 25 ms



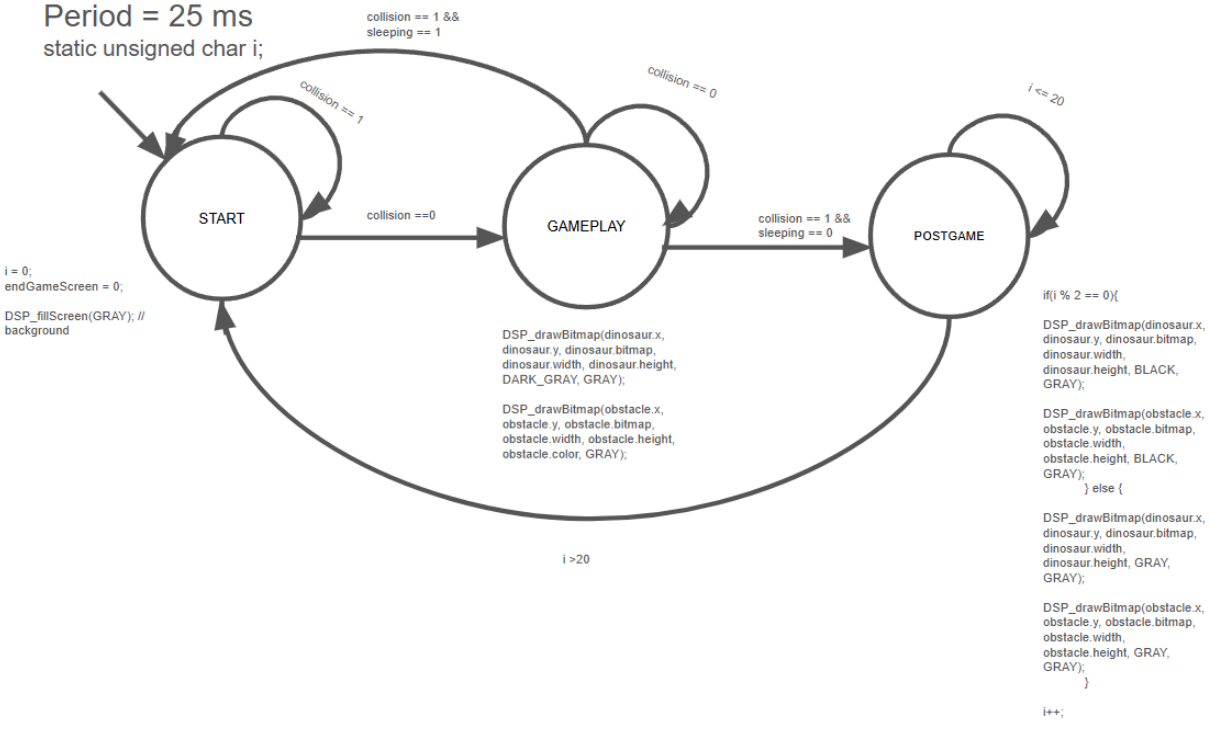
SCORE
Period = 50 ms



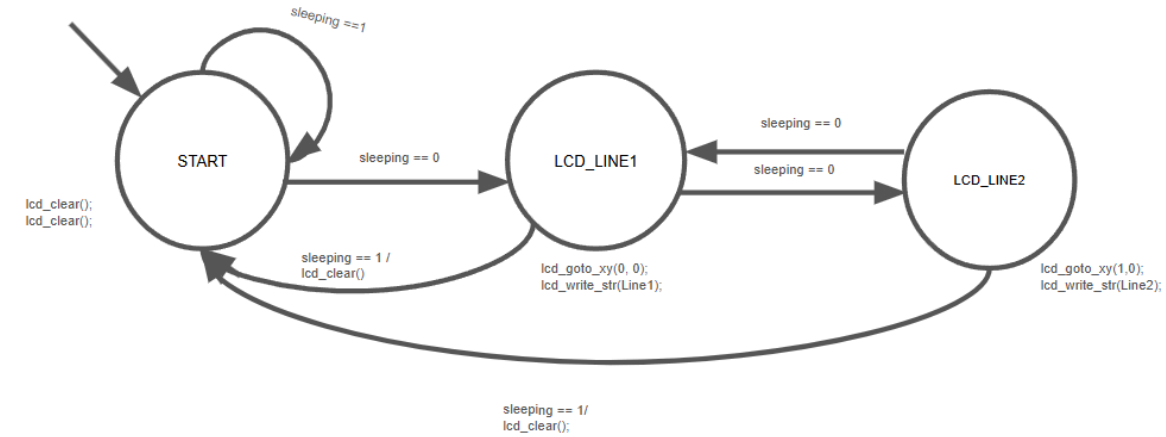
STRING
Period = 50 ms



DISPLAY
Period = 25 ms
static unsigned char i;

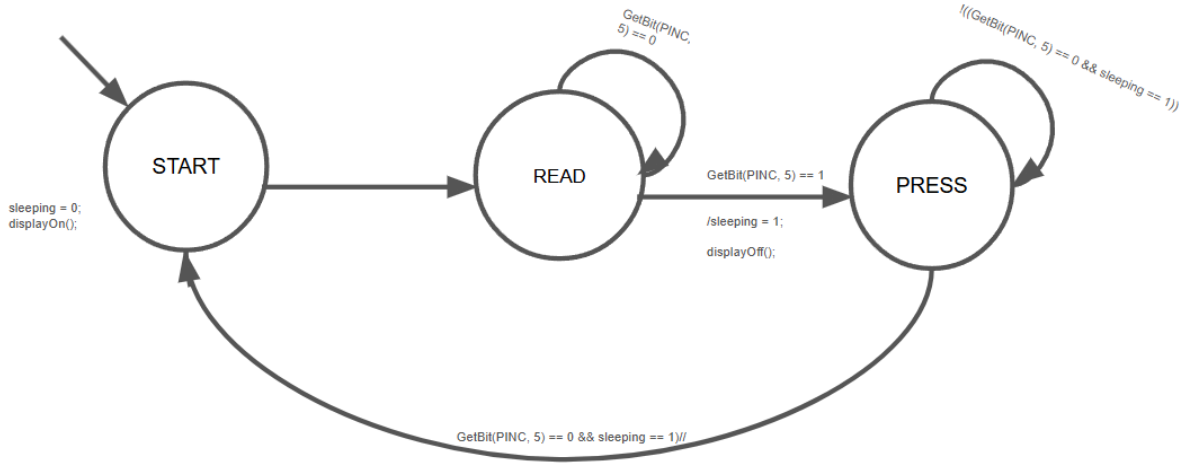


LCD
Period = 50 ms



SLEEP

Period = 25 ms



set_sleep_mode(SLEEP_MODE_PWR_DOWN); // Deepest sleep - wakes on external interrupt

interrupt_ON(); // Allow for the right button to act as an interrupt to wake the system

sleep_enable(); // Allow CPU to enter sleep

sleep_cpu(); // Enter sleep - CPU stops here until interrupt

sleep_disable(); // Resume after wake

interrupt_OFF(); // Stops the system from seeing the right button as an interrupt as a security measure

sleeping = 0;